P1120R0
Consistency improvements for <=> and other comparison operators

Published Proposal, 8 June 2018

This version:
http://wg21.link/p1120r0

Author:
Richard Smith (Google)

Audience:
CWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper provides wording to improve the consistency of <=> with other operators, by deprecating some cases where they diverge and tweaking the <=> rules, implementing part of P0946R0.

Table of Contents

1 Scope

2 Wording

2.1 D.x Arithmetic conversion on enumerations [depr.arith.conv.enum]

2.2 D.y Array comparisons [depr.array.comp]

References

Informative References

§ 1. Scope

This paper provides wording for the following changes as requested by Evolution in discussion of [P0946R0]:

- Deprecate usual arithmetic conversions between enumeration types and floating-point types
- Deprecate usual arithmetic conversions between two distinct enumeration types
- Deprecate two-way comparisons where both operators are array type
- Permit three-way comparison between unscoped enumeration types and integral types

This also applies the following changes as requested by Evolution, which are special cases of the broader changes to the usual arithmetic conversions:

- Deprecate two-way comparisons between enumeration types and floating-point types
- Deprecate two-way comparisons between two distinct enumeration types

§ 2. Wording

Add a new paragraph after [expr.arith.conv]p1:

> If one operand is of enumeration type and the other operand is of a different enumeration type or a floating-point type, this behavior is deprecated ([depr.arith.conv.enum]).

Change in [expr.spaceship]p4:

> If both operands have arithmetic types, or one operand has integral type and the other operand has unscoped enumeration type, the usual arithmetic conversions (8.3) are applied to the operands. Then: [...]

Change in [expr.rel]p1, splitting it into two paragraphs as shown:

> The relational operators group left-to-right. [*Example*: `a<b<c` means `(a<b)<c` and not `(a<b)&&(b<c)`. — *end example*] [... *relational-expression* grammar ...] The lvalue-to-rvalue ([conv.lval]), array-to-pointer ([conv.array]), and function-to-pointer ([conv.func]) standard conversions are performed on the operands. The comparison is deprecated if both operands were of array type prior to these conversions ([depr.array.comp]).
>
> The converted operands shall have arithmetic, enumeration, or pointer type. [...]

Change in [expr.eq]p1, splitting it into two paragraphs as shown.

> The `==` (equal to) and the `!=` (not equal to) operators group left-to-right. The lvalue-to-rvalue ([conv.lval]), array-to-pointer ([conv.array]), and function-to-pointer ([conv.func]) standard conversions are performed on the operands. The comparison is deprecated if both operands were of array type prior to these conversions.
>
> The converted operands shall have arithmetic, enumeration, pointer, or pointer-to-member type, or type `std::nullptr_t`. The operators `==` and `!=` both yield `true` or `false`, i.e., a result of type `bool`. In each case below, the operands shall have the same type after the specified conversions have been applied.

Add new subclauses to Annex D as follows:

2.1. D.x Arithmetic conversion on enumerations [depr.arith.conv.enum]

The ability to apply the usual arithmetic conversions ([expr.arith.conv]) on operands where one is of enumeration type and the other is of a different enumeration type or a floating-point type is deprecated. [*Note*: Three-way comparisons ([expr.spaceship]) between such operands are ill-formed.]

[*Example*:

```
enum E1 { e };
enum E2 { f };
bool b = e <= 3.7;    // deprecated
int k = f - e;        // deprecated
auto cmp = e <=> f;   // ill-formed
```

]

2.2. D.y Array comparisons [depr.array.comp]

Equality and relational comparisons ([expr.eq], [expr.rel]) between two operands of array type are deprecated. [*Note*: Three-way comparisons ([expr.spaceship]) between such operands are ill-formed.]

[*Example*:

```
int arr1[5];
int arr2[5];
bool same = arr1 == arr2;   // deprecated, same as &arr[0] == &arr[1],
                            // does not compare array contents
auto cmp = arr1 <=> arr2;   // ill-formed
```

]

§ References

§ Informative References

[P0946R0]

Richard Smith. Towards consistency between <=> and other comparison operators. URL: https://wg21.link/p0946r0